Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
Wydział Informatyki

Algorytmy macierzowe

Rekurencyjne mnożenie macierzy

Metody Bineta, Strassena i Ladermana — sprawozdanie z zadania 1

Piotr Szamocki · Mateusz Stoch

Kraków, październik 2026

Spis treści

1. Cel zadania 2
2. Język i środowisko 2
3. Implementacja 2
4. Pseudokody 4
5. Najważniejsze fragmenty kodu 7
6. Złożoność obliczeniowa 10
7. Wyniki pomiarów 11
8. Porównanie z Octave 13
9. Wnioski 14

1. Cel zadania

Zaimplementowaliśmy trzy rekurencyjne algorytmy mnożenia macierzy:

1. metodę Bineta (bloki 2×2, 8 mnożeń bloków),
2. metodę Strassena (bloki 2×2, 7 mnożeń bloków),
3. metodę Ladermana (bloki 3×3, 23 mnożenia bloków).

Wymagania:

- macierze kwadratowe o dowolnym rozmiarze,
- elementy losowe z przedziału $(10^{-8}, 1)$,
- zliczanie operacji zmiennoprzecinkowych $(+ - * /)$.

Każdy algorytm ma wersję jednowątkową i wielowątkową. Obie wersje wykonują te same operacje.

2. Język i środowisko

2.1. Dlaczego Java nie jest językiem interpretowanym

Kompilator `javac` tłumaczy kod źródłowy na kod bajtowy. Podczas działania maszyna wirtualna (HotSpot) kompiluje często używane fragmenty do kodu maszynowego (kompilacja JIT). Pętle mnożenia macierzy wykonują się więc jako kod maszynowy.

Konsekwencja: pierwsze wywołania są wolniejsze, bo kod nie jest jeszcze skompilowany. Dlatego powtarzamy pomiary i uśredniamy wyniki (Rozdział 7.1).

2.2. Sprzęt

Tabela 1: Konfiguracja pomiarowa

Procesor	AMD Ryzen 7 7700 8-Core Processor (3.80 GHz)
Pamięć RAM	32,0 GB, DDR5, 6000MT/s
System operacyjny	Windows 11
JDK	OpenJDK 25

3. Implementacja

3.1. Macierz i widoki

Dane macierzy mają jednego właściciela: jedną tablicę 1D. Elementy leżą w niej wiersz po wierszu.

Algorytmy nie używają tej tablicy bezpośrednio. Używają **widoków**. Widok to prostokątny fragment tablicy. Opisują go cztery liczby:

- `offset` — indeks pierwszego elementu,
- `rows`, `cols` — wymiary,
- `stride` — przeskok między kolejnymi wierszami.

Element (i, j) widoku ma indeks

$$\text{offset} + i \cdot \text{stride} + j. \tag{1}$$

Widok na podblok jest nowym widokiem na tę samą tablicę. Zmienia się tylko `offset` i wymiary. `stride` się nie zmienia (Rysunek 1).

0	1	2	3	4	5
6	7	8	9	10	11
12	13	14	15	16	17
18	19	20	21	22	23
24	25	26	27	28	29
30	31	32	33	34	35

tablica 1D (indeksy 0–35):

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35

Rysunek 1: Macierz 6×6 w tablicy 1D i widok na blok 3×3 od pozycji $(1, 2)$: offset $= 8$, stride $= 6$.

Zalety:

- **Tani podział.** Podział na bloki nie kopiuje danych. Tworzy tylko nowe widoki.
- **Mało alokacji.** Wynik podproblemu trafia przez widok od razu do macierzy C.
- **Dobra lokalność.** Wiersz widoku jest ciągłym fragmentem pamięci.
- **Prosty kod.** Bloki 2×2 i 3×3 tworzy się jednym wywołaniem.

3.2. Zliczanie operacji

Program zlicza operacje podczas działania. Każda operacja podstawowa dodaje do licznika wartość z Tabela 2. Odejmowanie liczy się jako dodawanie.

Tabela 2: Operacje podstawowe i ich koszt (A: $m \times k$, B: $k \times n$; przy dodawaniu macierze mają wymiar $p \times q$)

Działanie	**Mnożenia**	**Dodawania**
$C := A \pm B$	0	pq
$C := AB$ (klasycznie)	mkn	$m(k-1)n$
$C := C + AB$ (klasycznie)	mkn	mkn

3.3. Wersja jedno- i wielowątkowa

Każdy algorytm ma dwie wersje:

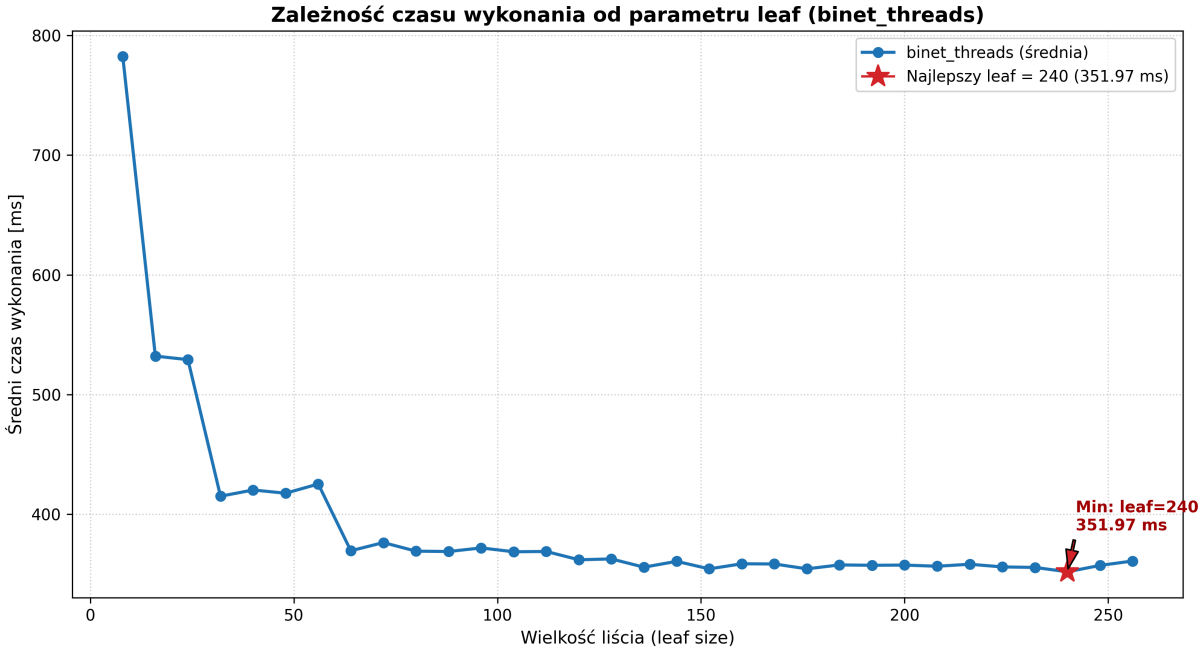
- **jednowątkową** — podproblemy wykonują się po kolei,
- **wielowątkową** — używa `ForkJoinPool` z liczbą wątków równą liczbie wątków procesora. Wszystkie podproblemy oprócz ostatniego trafiają do puli. Ostatni podproblem wykonuje bieżący wątek. Potem wątek czeka na pozostałe.

3.4. Podział macierzy

- **Binet** dzieli każdy wymiar na $\lfloor d/2 \rfloor$ i $\lceil d/2 \rceil$.
- **Strassen** dzieli wymiary na połowy. Nieparzysty wymiar → peeling (Algorytm 4).
- **Laderman** dzieli wymiary na trzy części. Niepodzielny wymiar → peeling (Algorytm 4).

3.5. Próg rekursji

Rekursja kończy się, gdy największy wymiar bloku jest $\leq n_0$ (parametr `leaf`). Taki blok mnożymy algorytmem klasycznym. Dla małych bloków koszt rekursji (widoki, zadania, bufory) jest większy niż zysk z mniejszej liczby mnożeń. Pomiary wykonano dla $n_0 = 240$.



Rysunek 2:

3.6. Weryfikacja poprawności

Dla każdego rozmiaru program liczy iloczyn referencyjny algorytmem klasycznym. Wynik każdego wariantu porównuje z referencją element po elemencie. Tolerancja wynosi 10^{-9}.

4. Pseudokody

Notacja. Bloki to widoki na macierz. Zapis do C_{TL} trafia od razu do C. W bloku **równolegle** każdy wiersz to osobne zadanie w wersji wielowątkowej. W wersji jednowątkowej wiersze wykonują się po kolei.

4.1. Metoda Bineta

$[C] := \text{Binet}(A, B, C)$ ▷ oblicza $C := C + AB$; na starcie $C = 0$

jeśli $\max(m, k, n) \le n_0$ **to**
  $C := C + AB$ ▷ klasycznie
  zwróć
koniec jeśli

$$A \to \left(\begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array}\right), B \to \left(\begin{array}{c|c} B_{TL} & B_{TR} \\ \hline B_{BL} & B_{BR} \end{array}\right), C \to \left(\begin{array}{c|c} C_{TL} & C_{TR} \\ \hline C_{BL} & C_{BR} \end{array}\right)$$

  gdzie bloki mają połowę wymiarów (zaokrąglenie w dół dla TL)

  równolegle
    $C_{TL} := C_{TL} + A_{TL}B_{TL}$; $C_{TL} := C_{TL} + A_{TR}B_{BL}$
    $C_{TR} := C_{TR} + A_{TL}B_{TR}$; $C_{TR} := C_{TR} + A_{TR}B_{BR}$
    $C_{BL} := C_{BL} + A_{BL}B_{TL}$; $C_{BL} := C_{BL} + A_{BR}B_{BL}$
    $C_{BR} := C_{BR} + A_{BL}B_{TR}$; $C_{BR} := C_{BR} + A_{BR}B_{BR}$
  koniec równolegle ▷ każdy iloczyn: wywołanie rekurencyjne Binet

Algorytm 1: Metoda Bineta

4.2. Metoda Strassena

$[C] := \text{Strassen}(A, B)$ ▷ oblicza $C := AB$

jeśli $\max(m,k,n) \leq n_0$ **lub** $\min(m,k,n) < 2$ **to**
  $C := AB$ ▷ klasycznie
  zwróć
koniec jeśli
jeśli któryś wymiar nie dzieli się przez 2 **to**
  $C := \text{Peeling}(A, B, 2, \text{Strassen})$ ▷ Algorytm 4
  zwróć
koniec jeśli
$A \to \left(\begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array}\right), B \to \left(\begin{array}{c|c} B_{TL} & B_{TR} \\ \hline B_{BL} & B_{BR} \end{array}\right), C \to \left(\begin{array}{c|c} C_{TL} & C_{TR} \\ \hline C_{BL} & C_{BR} \end{array}\right)$
  gdzie bloki mają połowę wymiarów

---

  równolegle
    $M_1 := \text{Strassen}(A_{TL} + A_{BR}, B_{TL} + B_{BR})$
    $M_2 := \text{Strassen}(A_{BL} + A_{BR}, B_{TL})$
    $M_3 := \text{Strassen}(A_{TL}, B_{TR} - B_{BR})$
    $M_4 := \text{Strassen}(A_{BR}, B_{BL} - B_{TL})$
    $M_5 := \text{Strassen}(A_{TL} + A_{TR}, B_{BR})$
    $M_6 := \text{Strassen}(A_{BL} - A_{TL}, B_{TL} + B_{TR})$
    $M_7 := \text{Strassen}(A_{TR} - A_{BR}, B_{BL} + B_{BR})$
  koniec równolegle

---

  równolegle
    $C_{TL} := M_1 + M_4 - M_5 + M_7$
    $C_{TR} := M_3 + M_5$
    $C_{BL} := M_2 + M_4$
    $C_{BR} := M_1 - M_2 + M_3 + M_6$
  koniec równolegle

Algorytm 2: Metoda Strassena

Jeden poziom: 7 mnożeń i 18 dodawań bloków.

4.3. Metoda Ladermana

Bloki 3×3 numerujemy od 1 do 3, jak w oryginalnym schemacie.

$[C] \coloneqq \text{Laderman}(A, B)$ ▷ oblicza $C \coloneqq AB$

jeśli $\max(m,k,n) \leq n_0$ **lub** $\min(m,k,n) < 3$ **to**
  $C \coloneqq AB$ ▷ klasycznie
  zwróć
koniec jeśli
jeśli któryś wymiar nie dzieli się przez 3 **to**
  $C \coloneqq \text{Peeling}(A, B, 3, \text{Laderman})$ ▷ Algorytm 4
  zwróć
koniec jeśli

$$A \to \left(\begin{array}{c|c|c} A_{11} & A_{12} & A_{13} \\ \hline A_{21} & A_{22} & A_{23} \\ \hline A_{31} & A_{32} & A_{33} \end{array}\right), B \to \left(\begin{array}{c|c|c} B_{11} & B_{12} & B_{13} \\ \hline B_{21} & B_{22} & B_{23} \\ \hline B_{31} & B_{32} & B_{33} \end{array}\right), C \to \left(\begin{array}{c|c|c} C_{11} & C_{12} & C_{13} \\ \hline C_{21} & C_{22} & C_{23} \\ \hline C_{31} & C_{32} & C_{33} \end{array}\right)$$

gdzie bloki mają jedną trzecią wymiarów

---

równolegle ▷ każdy iloczyn: wywołanie rekurencyjne Laderman

$M_1 \coloneqq (A_{11} + A_{12} + A_{13} - A_{21} - A_{22} - A_{32} - A_{33})B_{22}$
$M_2 \coloneqq (A_{11} - A_{21})(-B_{12} + B_{22})$
$M_3 \coloneqq A_{22}(-B_{11} + B_{12} + B_{21} - B_{22} - B_{23} - B_{31} + B_{33})$
$M_4 \coloneqq (-A_{11} + A_{21} + A_{22})(B_{11} - B_{12} + B_{22})$
$M_5 \coloneqq (A_{21} + A_{22})(-B_{11} + B_{12})$
$M_6 \coloneqq A_{11}B_{11}$
$M_7 \coloneqq (-A_{11} + A_{31} + A_{32})(B_{11} - B_{13} + B_{23})$
$M_8 \coloneqq (-A_{11} + A_{31})(B_{13} - B_{23})$
$M_9 \coloneqq (A_{31} + A_{32})(-B_{11} + B_{13})$
$M_{10} \coloneqq (A_{11} + A_{12} + A_{13} - A_{22} - A_{23} - A_{31} - A_{32})B_{23}$
$M_{11} \coloneqq A_{32}(-B_{11} + B_{13} + B_{21} - B_{22} - B_{23} - B_{31} + B_{32})$
$M_{12} \coloneqq (-A_{13} + A_{32} + A_{33})(B_{22} + B_{31} - B_{32})$
$M_{13} \coloneqq (A_{13} - A_{33})(B_{22} - B_{32})$
$M_{14} \coloneqq A_{13}B_{31}$
$M_{15} \coloneqq (A_{32} + A_{33})(-B_{31} + B_{32})$
$M_{16} \coloneqq (-A_{13} + A_{22} + A_{23})(B_{23} + B_{31} - B_{33})$
$M_{17} \coloneqq (A_{13} - A_{23})(B_{23} - B_{33})$
$M_{18} \coloneqq (A_{22} + A_{23})(-B_{31} + B_{33})$
$M_{19} \coloneqq A_{12}B_{21}$
$M_{20} \coloneqq A_{23}B_{32}$
$M_{21} \coloneqq A_{21}B_{13}$
$M_{22} \coloneqq A_{31}B_{12}$
$M_{23} \coloneqq A_{33}B_{33}$

koniec równolegle

---

równolegle

$C_{11} \coloneqq M_6 + M_{14} + M_{19}$
$C_{12} \coloneqq M_1 + M_4 + M_5 + M_6 + M_{12} + M_{14} + M_{15}$
$C_{13} \coloneqq M_6 + M_7 + M_9 + M_{10} + M_{14} + M_{16} + M_{18}$
$C_{21} \coloneqq M_2 + M_3 + M_4 + M_6 + M_{14} + M_{16} + M_{17}$
$C_{22} \coloneqq M_2 + M_4 + M_5 + M_6 + M_{20}$
$C_{23} \coloneqq M_{14} + M_{16} + M_{17} + M_{18} + M_{21}$
$C_{31} \coloneqq M_6 + M_7 + M_8 + M_{11} + M_{12} + M_{13} + M_{14}$
$C_{32} \coloneqq M_{12} + M_{13} + M_{14} + M_{15} + M_{22}$
$C_{33} \coloneqq M_6 + M_7 + M_8 + M_9 + M_{23}$

koniec równolegle

Algorytm 3: Metoda Ladermana

Jeden poziom: 23 mnożenia i 98 dodawań bloków (56 przy tworzeniu argumentów, 42 przy składaniu C).

4.3.1. Peeling

Peeling jest wspólny dla obu metod. Parametr q to liczba części podziału: $q = 2$ dla Strassena, $q = 3$ dla Ladermana. Gdy wymiar nie dzieli się przez q, odcinamy ostatnie wiersze (kolumny): najwyżej $q - 1$. Część główna idzie do rekursji danej metody. Paski brzegowe liczymy klasycznie.

$[C] \coloneqq \text{Peeling}(A, B, q, \text{Metoda})$ ▷ oblicza $C \coloneqq AB$

$m' \coloneqq m - (m \bmod q), \quad k' \coloneqq k - (k \bmod q), \quad n' \coloneqq n - (n \bmod q)$

$A \to \left(\begin{array}{c|c} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{array}\right), B \to \left(\begin{array}{c|c} B_{TL} & B_{TR} \\ \hline B_{BL} & B_{BR} \end{array}\right), C \to \left(\begin{array}{c|c} C_{TL} & C_{TR} \\ \hline C_{BL} & C_{BR} \end{array}\right)$

gdzie A_{TL} jest $m' \times k'$, B_{TL} jest $k' \times n'$, C_{TL} jest $m' \times n'$

równolegle

$C_{TL} \coloneqq \text{Metoda}(A_{TL}, B_{TL}); \quad C_{TL} \coloneqq C_{TL} + A_{TR}B_{BL}$

$\begin{pmatrix} C_{TR} \\ C_{BR} \end{pmatrix} \coloneqq A \begin{pmatrix} B_{TR} \\ B_{BR} \end{pmatrix}$

$C_{BL} \coloneqq (A_{BL} \,|\, A_{BR}) \begin{pmatrix} B_{TL} \\ B_{BL} \end{pmatrix}$

koniec równolegle ▷ paski: klasycznie

Algorytm 4: Peeling dla wymiarów niepodzielnych przez q (Strassen: $q = 2$, Laderman: $q = 3$)

5. Najważniejsze fragmenty kodu

5.1. Widok na tablicę

Indeks elementu i tworzenie widoku na podblok. Podblok dostaje tę samą tablicę i ten sam `stride`.

```
int idx(int i, int j) { return offset + i * stride + j; }

public MatrixView<A> block(int row0, int col0, int height, int width) {
    if (row0 < 0 || col0 < 0 || height < 0 || width < 0
            || row0 + height > rows || col0 + width > cols)
        throw new IndexOutOfBoundsException("Blok poza macierzą");
    return new MatrixView<>(data, idx(row0, col0), height, width, stride);
}
```

5.2. Zliczanie operacji

Liczniki rosną o wartość wyliczoną z wymiarów, raz na wywołanie.

```
// c = a * b (c nie może nachodzić na a ani b)
public final void multiply(MatrixView<A> a, MatrixView<A> b, MatrixView<A> c) {
    checkMul(a, b, c);
    zeroImpl(c);
    multiplyAddImpl(a, b, c);
    muls.add((long) a.rows * a.cols * b.cols);
    adds.add((long) a.rows * (a.cols - 1) * b.cols);  // k-1 dodawań na element
}

// c += a * b (c nie może nachodzić na a ani b)
public final void multiplyAdd(MatrixView<A> a, MatrixView<A> b, MatrixView<A> c) {
    checkMul(a, b, c);
    multiplyAddImpl(a, b, c);
    long n = (long) a.rows * a.cols * b.cols;
    muls.add(n);
```

```
    adds.add(n);
}
```

5.3. Mnożenie klasyczne (liście rekursji)

Kolejność pętli i–k–j. Pętla wewnętrzna idzie po ciągłym wierszu B i C.

```
protected void multiplyAddImpl(MatrixView<double[]> a, MatrixView<double[]> b,
MatrixView<double[]> c) {
    for (int i = 0; i < a.rows; i++)
        for (int k = 0; k < a.cols; k++) {
            double aik = a.data[a.idx(i, k)];
            int cBase = c.idx(i, 0), bBase = b.idx(k, 0);
            for (int j = 0; j < b.cols; j++)
                c.data[cBase + j] += aik * b.data[bBase + j];
        }
}
```

5.4. Binet: krok rekurencyjny

Podział na bloki to tylko widoki. Cztery zadania liczą cztery bloki C. Trzy trafiają do puli, czwarte liczy bieżący wątek.

```
int m1 = m / 2, k1 = k / 2, n1 = n / 2;
int m2 = m - m1, k2 = k - k1, n2 = n - n1;

var a11 = a.block(0, 0, m1, k1);   var a12 = a.block(0, k1, m1, k2);
var a21 = a.block(m1, 0, m2, k1);  var a22 = a.block(m1, k1, m2, k2);

var b11 = b.block(0, 0, k1, n1);   var b12 = b.block(0, n1, k1, n2);
var b21 = b.block(k1, 0, k2, n1);  var b22 = b.block(k1, n1, k2, n2);

var c11 = c.block(0, 0, m1, n1);   var c12 = c.block(0, n1, m1, n2);
var c21 = c.block(m1, 0, m2, n1);  var c22 = c.block(m1, n1, m2, n2);

Runnable t11 = () -> { binetMultiply(ops, a11, b11, c11, leaf, pool, accumulate);
binetMultiplyAdd(ops, a12, b21, c11, leaf, pool); };
// ... t12, t21, t22 analogicznie

if (pool != null) {
    var f11 = ForkJoinTask.adapt(t11).fork();
    var f12 = ForkJoinTask.adapt(t12).fork();
    var f21 = ForkJoinTask.adapt(t21).fork();
    t22.run();
    f21.join();
    f12.join();
    f11.join();
} else {
    t11.run();
    t12.run();
    t21.run();
    t22.run();
}
```

5.5. Strassen: bufory, zadanie i składanie wyniku

$M_1, \ldots, M_7$ to widoki jednego bufora. Sumy argumentów też leżą w jednym buforze. Dzięki temu na węzeł są tylko trzy alokacje.

```
// Bufory na wyniki M1–M7 (7 bloków jako widoki jednego bufora)
var mBuf = ops.newMatrix(7 * m2, n2).view();
var m1r = mBuf.block(0 * m2, 0, m2, n2);
var m2r = mBuf.block(1 * m2, 0, m2, n2);
```

```
var m3r = mBuf.block(2 * m2, 0, m2, n2);
var m4r = mBuf.block(3 * m2, 0, m2, n2);
var m5r = mBuf.block(4 * m2, 0, m2, n2);
var m6r = mBuf.block(5 * m2, 0, m2, n2);
var m7r = mBuf.block(6 * m2, 0, m2, n2);

// Arena na tymczasowe sumy: po jednym lA (5) i lB (5) na zadanie
var tA = ops.newMatrix(5 * m2, k2).view();
var tB = ops.newMatrix(5 * k2, n2).view();

// M1 = strassen(A11 + A22, B11 + B22)
Runnable t1 = () -> {
    var lA = tA.block(0 * m2, 0, m2, k2);
    var lB = tB.block(0 * k2, 0, k2, n2);
    ops.add(a11, a22, lA);
    ops.add(b11, b22, lB);
    strassenMultiply(ops, lA, lB, m1r, leaf, pool);
};
// ... t2–t7 analogicznie, potem fork/join jak w metodzie Bineta

// Składanie wyniku z M1–M7
Runnable s11 = () -> {
    // C11 = M1 + M4 - M5 + M7
    ops.add(m1r, m4r, c11);
    ops.subtract(c11, m5r, c11);
    ops.add(c11, m7r, c11);
};
// ... C12, C21, C22 analogicznie
```

5.6. Peeling (Strassen)

Część parzysta idzie do rekursji. Paski brzegowe liczymy klasycznie, równolegle z częścią główną. Laderman ma ten sam kod z `% 3`.

```
int m1 = m - m % 2;
int k1 = k - k % 2;
int n1 = n - n % 2;

if (m1 != m || k1 != k || n1 != n) {
    if (pool != null) {
        ForkJoinTask<?> fRight = null;
        ForkJoinTask<?> fBottom = null;

        if (n1 < n) {
            fRight = ForkJoinTask.adapt(() -> {
                ops.multiply(a, b.block(0, n1, k, n - n1), c.block(0, n1, m, n - n1));
            }).fork();
        }

        if (m1 < m) {
            fBottom = ForkJoinTask.adapt(() -> {
                ops.multiply(a.block(m1, 0, m - m1, k), b.block(0, 0, k, n1), c.block(m1, 0, m -
m1, n1));
            }).fork();
        }

        strassenMultiply(ops, a.block(0, 0, m1, k1), b.block(0, 0, k1, n1), c.block(0, 0, m1, n1),
leaf, pool);

        if (k1 < k) {
            ops.multiplyAdd(a.block(0, k1, m1, k - k1), b.block(k1, 0, k - k1, n1), c.block(0, 0,
m1, n1));
        }

        if (fBottom != null) fBottom.join();
```

```
        if (fRight != null) fRight.join();
    } else {
        // ... to samo sekwencyjnie
    }
    return;
}
```

6. Złożoność obliczeniowa

Oznaczenia dla macierzy $n \times n$:

- $M(n)$ — liczba mnożeń,
- $D(n)$ — liczba dodawań (z odejmowaniami).

Algorytm klasyczny: $M(n) = n^3$, $D(n) = n^3 - n^2$.

6.1. Binet

Rekursja zmienia tylko kolejność sumowania. Każdy iloczyn $a_{il}b_{lj}$ liczymy raz. Dlatego:

$$M(n) = n^3, \quad D(n) = n^3 - n^2. \tag{2}$$

To samo co w algorytmie klasycznym, dla każdego n i n_0.

Złożoność obliczeniowa: $O(n^3)$.

6.2. Strassen

Jeden poziom: 7 wywołań na blokach $n/2$ i 18 dodawań bloków:

$$M(n) = 7M\left(\frac{n}{2}\right), \quad D(n) = 7D\left(\frac{n}{2}\right) + 18\left(\frac{n}{2}\right)^2. \tag{3}$$

Dla $n = 2^k$ i rekursji do bloków 1×1:

$$M(n) = 7^k = n^{\log_2 7}, \quad D(n) = 6\left(n^{\log_2 7} - n^2\right). \tag{4}$$

Złożoność obliczeniowa: $O\left(n^{\log_2 7}\right) \approx O(n^{2.81})$.

6.3. Laderman

Jeden poziom: 23 wywołania na blokach $n/3$ i 98 dodawań bloków:

$$M(n) = 23M\left(\frac{n}{3}\right), \quad D(n) = 23D\left(\frac{n}{3}\right) + 98\left(\frac{n}{3}\right)^2. \tag{5}$$

Dla $n = 3^k$ i rekursji do bloków 1×1:

$$M(n) = 23^k = n^{\log_3 23}, \quad D(n) = 7\left(n^{\log_3 23} - n^2\right). \tag{6}$$

Złożoność obliczeniowa: $O\left(n^{\log_3 23}\right) \approx O(n^{2.85})$.

6.4. Wpływ peelingu i progu n_0

W każdym węźle rekursji Strassen i Laderman i tak dodają bloki. Koszt tych dodawań jest rzędu n^2 (Strassen: $18(n/2)^2$, Laderman: $98(n/3)^2$).

Peeling dodaje w węźle mnożenie pasków o szerokości 1 lub 2. Pasek ma około n wierszy i n kolumn, więc jego koszt też jest rzędu n^2. Rekurencja ma więc tę samą postać co bez peelingu:

$$T(n) = 7T\left(\frac{n}{2}\right) + cn^2 \quad \text{(Strassen)}, \qquad T(n) = 23T\left(\frac{n}{3}\right) + cn^2 \quad \text{(Laderman)}. \tag{7}$$

Peeling zwiększa tylko stałą c.

O złożoności decyduje liczba wywołań rekurencyjnych (7 lub 23), a nie praca cn^2 w węźle. Wykładniki $\log_2 7 \approx 2.81$ i $\log_3 23 \approx 2.85$ są większe od 2, więc składnik cn^2 nie zmienia rzędu. Z peelingiem złożoność to nadal $O\big(n^{\log_2 7}\big)$ i $O\big(n^{\log_3 23}\big)$.

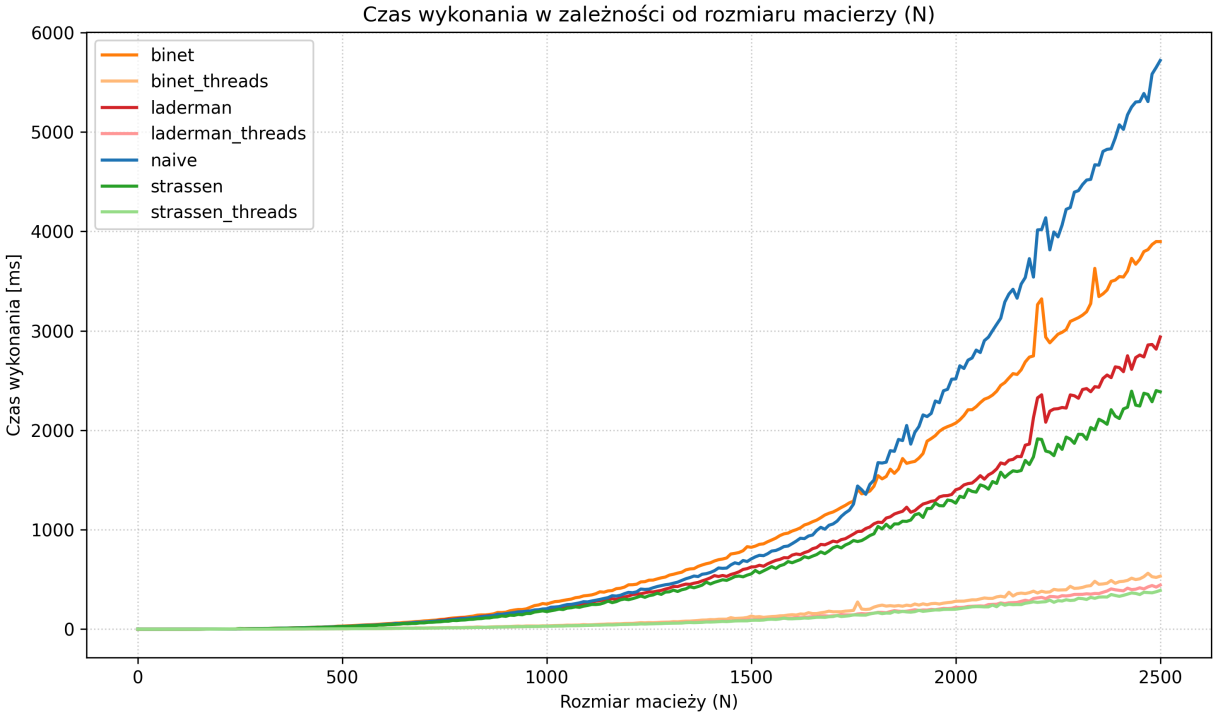
Próg n_0 działa podobnie. Rekursja kończy się na blokach $n_0 \times n_0$, które liczymy klasycznie. Zmieniają się stałe, a nie złożoność.

7. Wyniki pomiarów

7.1. Metodyka

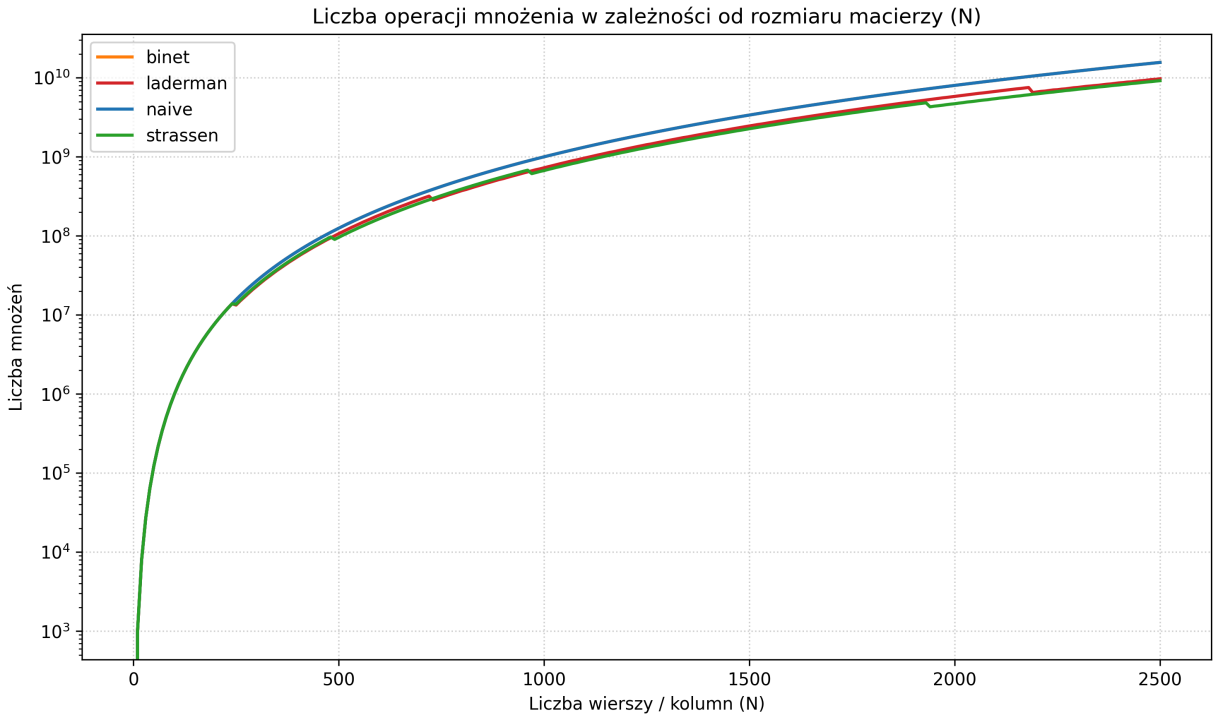
- Rozmiary: $n = 10, 20, 30, ..., 2500$ (krok 10).
- Wartości elementów: losowe z $(10^{-8}, 1)$.
- Próg rekursji: $n_0 = 240$.
- Każdy wariant: czas, liczba mnożeń, liczba dodawań.
- 3 przebiegi, czasy uśrednione.

7.2. Czas działania



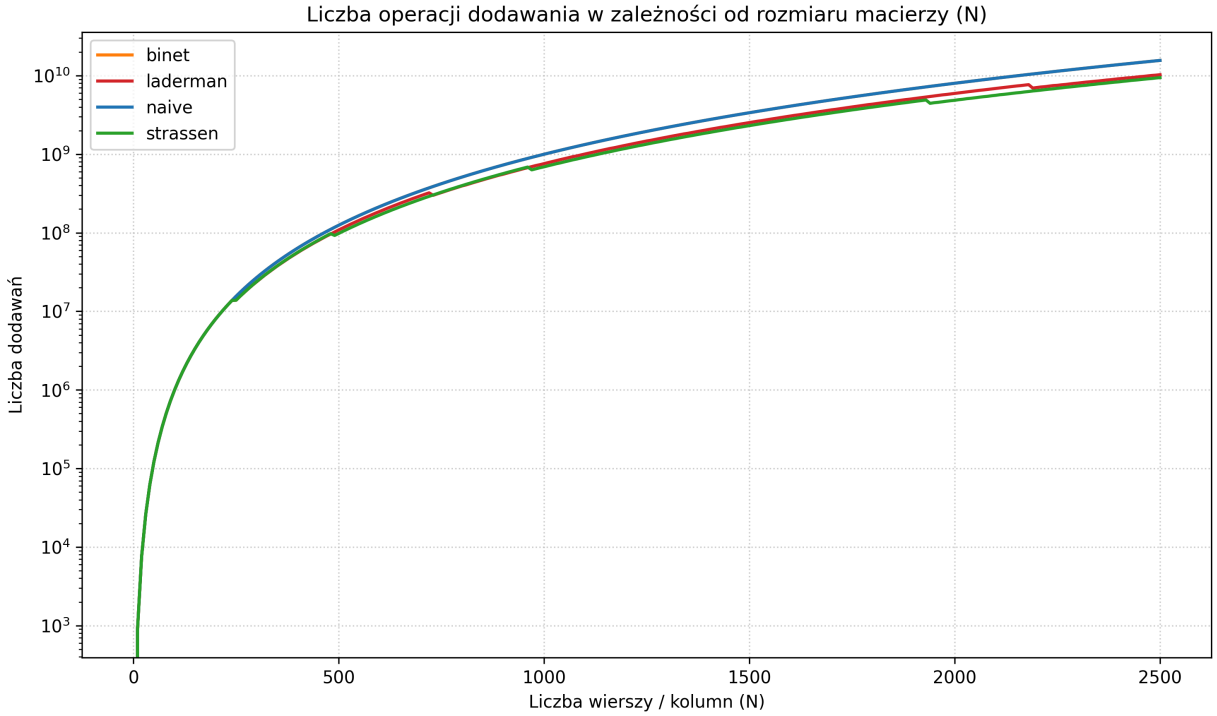
Rysunek 3: Czas mnożenia macierzy $n \times n$

7.3. Liczba mnożeń



Rysunek 4: Liczba mnożeń zmiennoprzecinkowych

7.4. Liczba dodawań



Rysunek 5: Liczba dodawań zmiennoprzecinkowych

8. Porównanie z Octave

Procedura:

1. Wylosowaliśmy A, B o rozmiarze 4×4 z przedziału $(10^{-8}, 1)$.
2. Pomnożyliśmy je każdą metodą z $n_0 = 1$, w wersji jedno- i wielowątkowej. Przy dużym n_0 rekursja by się nie wykonała.
3. Wypisaliśmy macierze z pełną precyzją (`%.17g`) i porównaliśmy wyniki z `A * B` w Octave 8.4.0:

```
A = load('A.txt');  B = load('B.txt');  C_java = load('C_java.txt');
C = A * B;
max_blad  = max(abs(C(:) - C_java(:)))
wzgl_blad = norm(C - C_java, 'fro') / norm(C, 'fro')
```

Dane i wynik (zaokrąglone do 4 miejsc):

$$A = \begin{pmatrix} 0.6186 & 0.8492 & 0.7904 & 0.5296 \\ 0.8417 & 0.1483 & 0.8438 & 0.5848 \\ 0.1667 & 0.7158 & 0.5393 & 0.2795 \\ 0.0557 & 0.0131 & 0.3578 & 0.7500 \end{pmatrix}, \quad B = \begin{pmatrix} 0.5786 & 0.2522 & 0.5377 & 0.5431 \\ 0.0891 & 0.2529 & 0.2611 & 0.3720 \\ 0.8833 & 0.9669 & 0.1329 & 0.0050 \\ 0.8571 & 0.8844 & 0.5446 & 0.5092 \end{pmatrix} \tag{8}$$

$$C = AB = \begin{pmatrix} 1.5857 & 1.6033 & 0.9478 & 0.9255 \\ 1.7467 & 1.5828 & 0.9219 & 0.8143 \\ 0.8762 & 0.9917 & 0.5004 & 0.5018 \\ 0.9922 & 1.0266 & 0.4893 & 0.4188 \end{pmatrix} \tag{9}$$

Wszystkie metody dają tę samą macierz C z dokładnością do 4 miejsc. Różnice w pełnej precyzji są w tabeli 3.

Tabela 3: Porównanie z Octave, macierz 4×4, $n_0 = 1$ ($\varepsilon \approx 2.2 \cdot 10^{-16}$)

Metoda	**Mnożenia / dodawania**	**Maks. błąd bezwzględny**	**Błąd względny**
Binet	64 / 48	0	0
Strassen	49 / 198	$8.9 \cdot 10^{-16}$	$3.2 \cdot 10^{-16}$
Laderman	60 / 128	$2.2 \cdot 10^{-16}$	$1.1 \cdot 10^{-16}$

Wersje jedno- i wielowątkowe dały identyczne wyniki.

Binet zgadza się z Octave co do bitu. Strassen i Laderman różnią się o kilka ostatnich bitów (najwyżej 6 jednostek ostatniego miejsca). Powód: wykonują więcej dodawań i odejmowań, więc błędy zaokrągleń są inne. Błąd względny jest rzędu ε. Wyniki są poprawne.

9. Wnioski

1. Binet wykonuje tyle samo operacji co algorytm klasyczny. Złożoność: $O(n^3)$.
2. Strassen i Laderman wykonują mniej mnożeń bloków (7 zamiast 8, 23 zamiast 27). Złożoność: $O(n^{2.81})$ i $O(n^{2.85})$. Kosztem jest więcej dodawań i dodatkowe bufory.
3. Peeling pozwala mnożyć macierze o dowolnym rozmiarze. Nie zmienia złożoności.
4. Widoki na tablicę 1D dają tani podział na bloki bez kopiowania danych.
5. Wyniki wszystkich metod zgadzają się z Octave z dokładnością do błędu zaokrągleń.
6. Algorytm Strassena okazał się najszybszą metodą spośród wszystkich testowanych.
7. Zrównoleglenie obliczeń za pomocą ForkJoinPool przyniosło stabilne, ponad 6-krotne przyspieszenie (średnio 6,3x dla Strassena i 7,2x dla Bineta) względem wersji jednowątkowej tego samego algorytmu.